

Gráficos - SEN0344 - V1.1

01/03/2026 - Lara Ribeiro

Descrição

Esta documentação acompanha o desenvolvimento do código MATLAB para auxiliar nos testes do sensor SEN0344 v2. Agora o arduino IDE tira todos os valores do sensor, é feito um gráfico no matlab e só depois é que os dados são filtrados e é criado um novo gráfico no MATLAB. Todos os gráficos e respetivos valores são guardados.

Quando o ficheiro acaba em _0 significa que os dados não estão filtrados e quando acaba em _1 significa que os dados foram filtrados.

CÓDIGO UTILIZADO DO ARDUINO IDE

Com o sensor sempre ligado a tirar medições de x em x ms

```
#include "DFRobot_BloodOxygen_S.h"

#define I2C_COMMUNICATION //use I2C for communication, but use the serial port
for communication if the line of codes were masked

#ifdef I2C_COMMUNICATION
#define I2C_ADDRESS 0x57
  DFRobot_BloodOxygen_S_I2C MAX30102(&Wire ,I2C_ADDRESS);
#else
  #if defined(ARDUINO_AVR_UNO) || defined(ESP8266)
    SoftwareSerial mySerial(4, 5);
    DFRobot_BloodOxygen_S_SoftWareUart MAX30102(&mySerial, 9600);
  #else
    DFRobot_BloodOxygen_S_HardWareUart MAX30102(&Serial1, 9600);
  #endif
#endif

#define I2C_SDA 8
#define I2C_SCL 9

void setup() {
  Wire.begin(I2C_SDA, I2C_SCL);
  Serial.begin(9600);

  while (false == MAX30102.begin())
  {
    // Adicionado prefixo ERR, para o MATLAB ignorar esta linha nos dados
    Serial.println("ERR, init fail!");
    delay(1000);
  }
  // Adicionados prefixos INIT, e START, para aparecerem na consola do MATLAB
```

```

    Serial.println("INIT, success!");
    Serial.println("START, measuring...");
}

void loop()
{
    // 1. LIGA O SENSOR
    MAX30102.sensorStartCollect();

    MAX30102.getHeartbeatSP02();

    int spo2 = MAX30102._sHeartbeatSP02.SP02;
    int hr = MAX30102._sHeartbeatSP02.Heartbeat;
    unsigned long tempo_ms = millis();

    Serial.print("DATA,");
    Serial.print(tempo_ms);
    Serial.print(",");
    Serial.print(spo2);
    Serial.print(",");
    Serial.println(hr);

    // 4. FICA DESLIGADO POR x mSEGUNDOS
    delay();
}

```

Sensor com duty cycle de x a tirar medições de x em x ms

```

#include "DFRobot_BloodOxygen_S.h"

#define I2C_COMMUNICATION //use I2C for communication, but use the serial port
for communication if the line of codes were masked

#ifdef I2C_COMMUNICATION
#define I2C_ADDRESS 0x57
    DFRobot_BloodOxygen_S_I2C MAX30102(&Wire ,I2C_ADDRESS);
#else
    #if defined(ARDUINO_AVR_UNO) || defined(ESP8266)
        SoftwareSerial mySerial(4, 5);
        DFRobot_BloodOxygen_S_SoftWareUart MAX30102(&mySerial, 9600);
    #else
        DFRobot_BloodOxygen_S_HardWareUart MAX30102(&Serial1, 9600);
    #endif
#endif

#define I2C_SDA 8
#define I2C_SCL 9

void setup() {

```

```
Wire.begin(I2C_SDA, I2C_SCL);
Serial.begin(9600);

while (false == MAX30102.begin())
{
    // Adicionado prefixo ERR, para o MATLAB ignorar esta linha nos dados
    Serial.println("ERR, init fail!");
    delay(1000);
}
// Adicionados prefixos INIT, e START, para aparecerem na consola do MATLAB
Serial.println("INIT, success!");
Serial.println("START, measuring...");
}

void loop()
{
    // 1. LIGA O SENSOR
    MAX30102.sensorStartCollect();
    Serial.println("INIT, Sensor ON - A recolher amostras...");

    // Guardamos o momento em que o sensor ligou
    unsigned long startON = millis();

    // 2. CICLO DE LEITURA (Executa enquanto não passarem Xms)
    while (millis() - startON < X)
    {
        MAX30102.getHeartbeatSP02();

        int spo2 = MAX30102._sHeartbeatSP02.SP02;
        int hr = MAX30102._sHeartbeatSP02.Heartbeat;
        unsigned long tempo_ms = millis();

        Serial.print("DATA,");
        Serial.print(tempo_ms);
        Serial.print(",");
        Serial.print(spo2);
        Serial.print(",");
        Serial.println(hr);

        // Pequena pausa entre amostras para não inundar o MATLAB
        // leituras de X em X ms
        delay(X);
    }

    // 3. DESLIGA O SENSOR
    MAX30102.sensorEndCollect();
    Serial.println("INIT, Sensor OFF - Descanso...");

    // 4. FICA DESLIGADO POR x mSEGUNDOS
    delay();
}
```

CÓDIGO UTILIZADO no MATLAB para obter valores do SEN0344

```
%% ===== CONFIGURAÇÃO =====
port = "COM20"; % <-- muda para a tua porta (ex.: "COM7")
baud = 9600; % tem de bater com o Serial.begin do ESP32
duracao_s = 600; % tempo de medicao
% =====

% Libertar a porta (se estiver presa no MATLAB)
delete(serialportfind("Port", port));

% Abrir serial
s = serialport(port, baud);
configureTerminator(s, "CR/LF"); % robusto no Windows com \r\n
flush(s);
t0 = tic;

% Variáveis para guardar os dados
time_ms = [];
SPO2_data = [];
HR_data = [];

fprintf("\nLeitura SEN0344 (SpO2 e HR) - Porta %s @ %d baud\n", port, baud);
fprintf("%-10s | %-10s | %-10s\n", "Tempo(s)", "SpO2 (%)", "HR (bpm)");
fprintf("%s\n", repmat("-", 1, 38));

while toc(t0) < duracao_s
    if s.NumBytesAvailable == 0
        pause(0.01);
        continue;
    end
    line = strtrim(string(readline(s)));

    % Espera o formato: DATA,ms,spo2,hr
    if startsWith(line, "DATA,")
        parts = split(line, ",");
        if numel(parts) == 4
            ms = str2double(parts(2));
            spo2 = str2double(parts(3));
            hr = str2double(parts(4));

            % Ignorar a linha se houver erro na conversão
            if isnan(ms) || isnan(spo2) || isnan(hr)
                continue;
            end

            % Guardar dados
            time_ms(end+1,1) = ms;          %#ok<SAGROW>
            SPO2_data(end+1,1) = spo2;      %#ok<SAGROW>
            HR_data(end+1,1) = hr;          %#ok<SAGROW>

            % Tempo relativo ao primeiro ponto em segundos
```

```

        if numel(time_ms) == 1
            tempo_s = 0;
        else
            tempo_s = (ms - time_ms(1)) / 1000;
        end

        % Output na consola
        fprintf("%-10.1f | %-10.2f | %-10.2f\n", tempo_s, spo2, hr);
    end

    elseif startsWith(line, "INIT,") || startsWith(line, "START") ||
startsWith(line, "HEADER") || startsWith(line, "ERR,")
        % Mostrar mensagens de estado do ESP32 sem poluir a tabela
        fprintf("[ESP32] %s\n", line);
    end
end

% Fechar porta
clear s
fprintf("%s\n", repmat("-", 1, 38));
fprintf(" Medição terminada. Pontos guardados: %d\n\n", numel(SPO2_data));

if isempty(SPO2_data)
    error("Não foi guardado nenhum ponto. Confirma que o ESP32 está a enviar
linhas no formato 'DATA,ms,spo2,hr'.");
end

% Converter tempo absoluto para segundos relativos ao início
t = (time_ms - time_ms(1)) / 1000;

%% ===== GRÁFICO FINAL =====
%load(" ")
figure("Name","SEN0344 - Oximetria e Frequência Cardíaca");

% Eixo Y Esquerdo (SpO2)
yyaxis left;
plot(t, SPO2_data, '-x', 'LineWidth', 1.5);
ylabel("SpO2 (%)");
ylim([85 100]); % Descomenta esta linha se quiseres fixar o limite do SpO2 entre
80 e 100%

% Eixo Y Direito (HR - Frequência Cardíaca)
yyaxis right;
plot(t, HR_data, '-x', 'LineWidth', 1.5);
ylabel("Heart Rate (bpm)");
ylim([30 200]);

% Configurações gerais do gráfico
grid on;
xlabel("Tempo (s)");
title("Monitorização de SpO2 e Frequência Cardíaca");
legend("SpO2 (%)", "Heart Rate (bpm)", "Location", "best");

%% ===== GUARDAR SEM FILTRO =====

```

```

save("teste_0")

%% ===== FILTRAGEM =====

[t_spo2, SPO2_clean] = remover_erros(t, SPO2_data);
[t_hr, HR_clean] = remover_erros(t, HR_data);

function [t_limpo, dados_limpos] = remover_erros(t_original, dados_originais)
    % A função identifica os índices onde o valor não é -1
    indices_validos = (dados_originais ~= -1);

    % Retorna os vetores apenas com os pontos válidos
    t_limpo = t_original(indices_validos);
    dados_limpos = dados_originais(indices_validos);
end

% ===== GRÁFICO FINAL =====
figure("Name","SEN0344 - Oximetria e Frequência Cardíaca");

% Eixo Y Esquerdo (SpO2)
yyaxis left;
plot(t_spo2, SPO2_clean, '-x', 'LineWidth', 1.5);
ylabel("SpO2 (%)");
ylim([90 100]); % Limite fixo entre 90% e 100%

% Eixo Y Direito (HR - Frequência Cardíaca)
yyaxis right;
plot(t_hr, HR_clean, '-x', 'LineWidth', 1.5);
ylabel("Heart Rate (bpm)");
ylim([30 200]); % Limite fixo entre 30 e 200 bpm

% Configurações gerais do gráfico
grid on;
xlabel("Tempo (s)");
title("Monitorização de SpO2 e Frequência Cardíaca");
legend("SpO2 (%)", "Heart Rate (bpm)", "Location", "best");

%% ===== GUARDAR COM FILTRO =====
save("teste_1")

```

CÓDIGO UTILIZADO no MATLAB para comparar valores entre o SEN e o PPG

```

%% ECG --> BPM
path = 'C:\Users\Lara\Desktop\3ano\PEE\MatLabPEE\TestesIEETA\ieetaX\';
filename = 'ecgX.txt';
matrix = readmatrix([path filename]);

ecg_data = matrix(:, 3);
flag = find(matrix(:,2) > 0)/1000;
inicio = flag(1) - 30;

```

```

flag = flag - inicio;
fim = flag(end) + 30;

% ecg_data = ecg_data((inicio*1000):(fim*1000));
ecg_data = ecg_data(max(1, round(inicio*1000)) : min(end, round((fim +
inicio)*1000)));

Fs = 1000; % Frequência de amostragem (1000 Hz)

N = length(ecg_data);

t_ecg = (0 : N-1)' / Fs;

ecg_limpo = detrend(ecg_data);

limite_altura = max(ecg_limpo) * 0.5;
[picos, locs] = findpeaks(ecg_limpo, t_ecg, 'MinPeakHeight', limite_altura,
'MinPeakDistance', 0.3);

rr_intervals = diff(locs);
bpm_irregular = 60 ./ rr_intervals;
t_bpm_irregular = locs(2:end);

t_1s = ceil(t_bpm_irregular(1)) : 1 : floor(t_bpm_irregular(end));

bpm_1s = interp1(t_bpm_irregular, bpm_irregular, t_1s, 'linear');

save('C:\Users\Lara\Desktop\3ano\PEE\MatLabPEE\TestesIEETA\ieetaX\ECGtoBPMX")

%% FIGURE COM GRÁFICO ECG VS SEN0344

load('C:\Users\Lara\Desktop\3ano\PEE\MatLabPEE\TestesSEN0344\IEETAX_1.mat")
load('C:\Users\Lara\Desktop\3ano\PEE\MatLabPEE\TestesIEETA\ieetaX\ECGtoBPMX.mat")
figure('Name','BPM - ECG VS SEN0344");

subplot(2,1,1); % ECG -> BPM
plot(t_1s, bpm_1s, '-x', 'LineWidth', 1.5);
title('Ritmo Cardíaco Interpolado (1 Amostra/s) -- ECG -> BPM ');
xlabel('Tempo (s)');
ylabel('Heart Rate (BPM)');
xlim([0 fim]);
grid on;
hold on;
yline(mean(bpm_1s), 'LineWidth', 2, 'Color', 'r');
xline(flag, 'LineWidth', 2, 'Color', 'g');
xline(fim, 'LineWidth', 2, 'Color', 'y');
hold off;
legend("BPM", "meanValue" );

subplot(2,1,2); % BPM DO SEN
plot(t_hr, HR_clean, '-x', 'LineWidth', 1.5);
title('BPM - SEN0344')
xlabel('Tempo (s)');

```

```

ylabel('Heart Rate (BPM)');
ylim([30 200]); % Limite fixo entre 30 e 200 bpm
xlim([0 fim]);

grid on;
hold on;
yline(mean(HR_clean(t_hr <= fim)), 'LineWidth', 2 , 'Color', 'r');
hold off;
legend("BPM", "meanValue" );

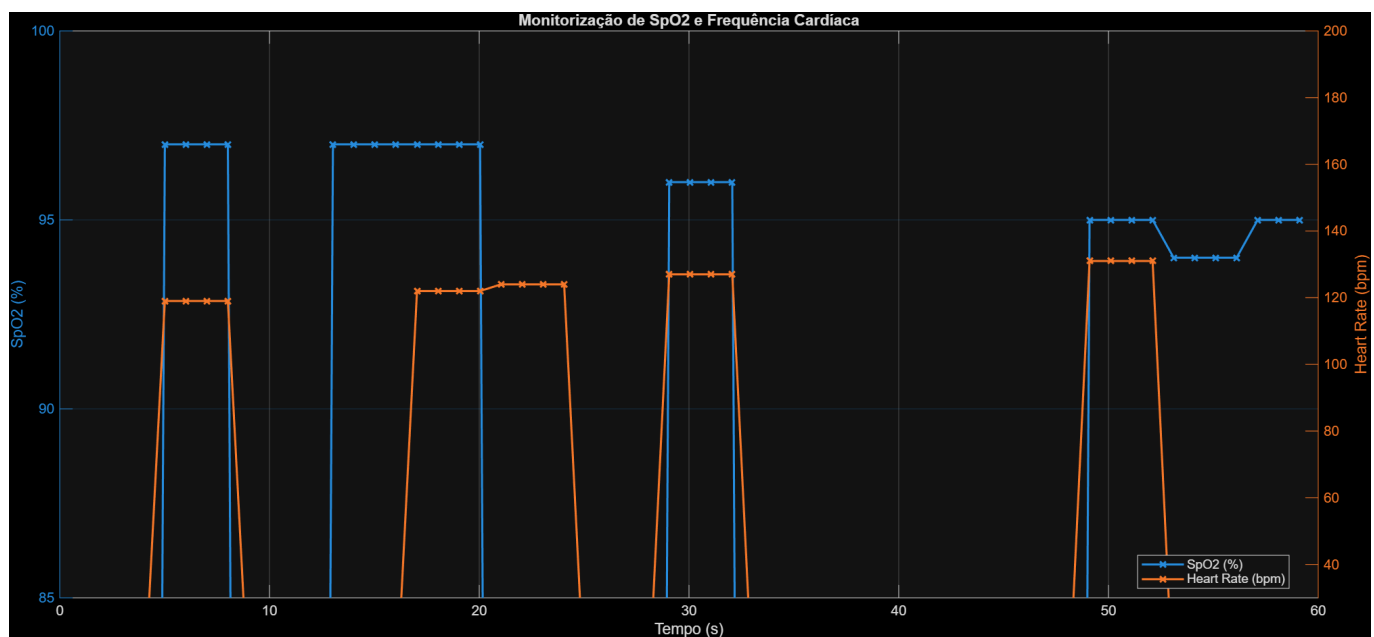
save("C:\Users\Lara\Desktop\3ano\PEE\MatLabPEE\TestesIEETA\ieetaX\BPM_ECG_VS_SEN0344_X");

```

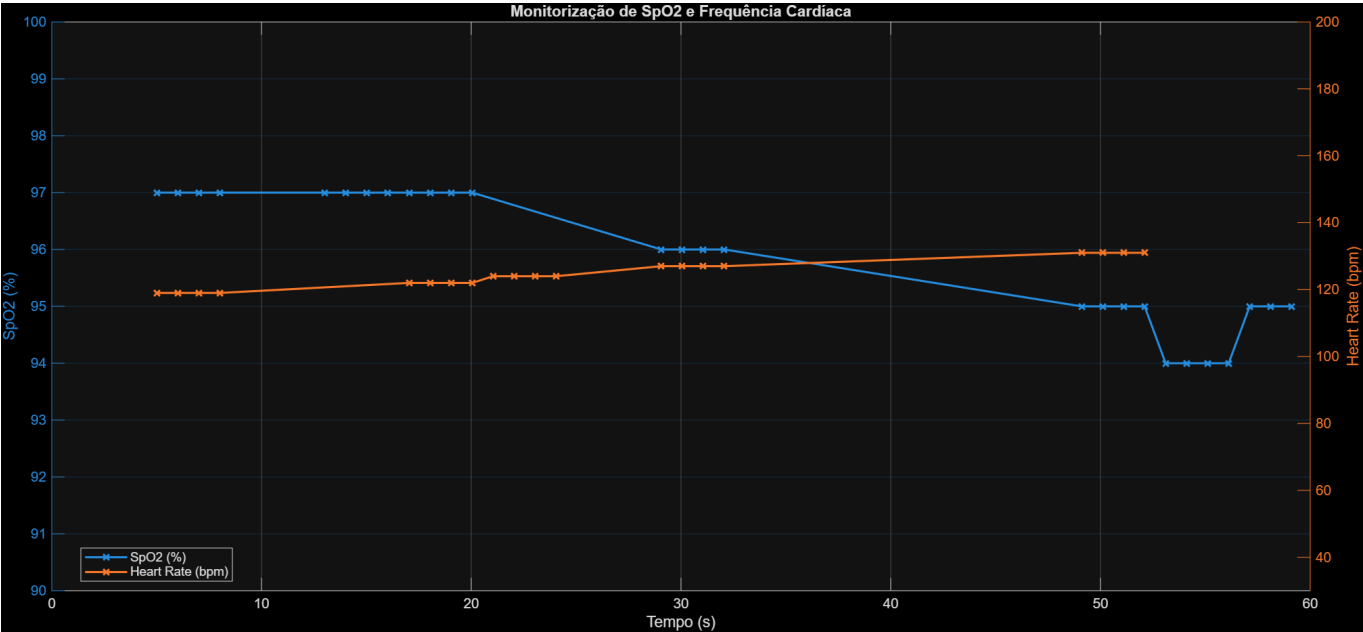
GRÁFICOS PRA VER SE O SENSOR AO ESTAR MUITO TEMPO LIGADO NAO SATURA

Esterno, sem desligar durante o tempo medido, dados retirados de 1 em 1s, entre cada teste a esp32 foi reiniciada Medidos numa mulher de 20 anos saudável, em repouso, parada

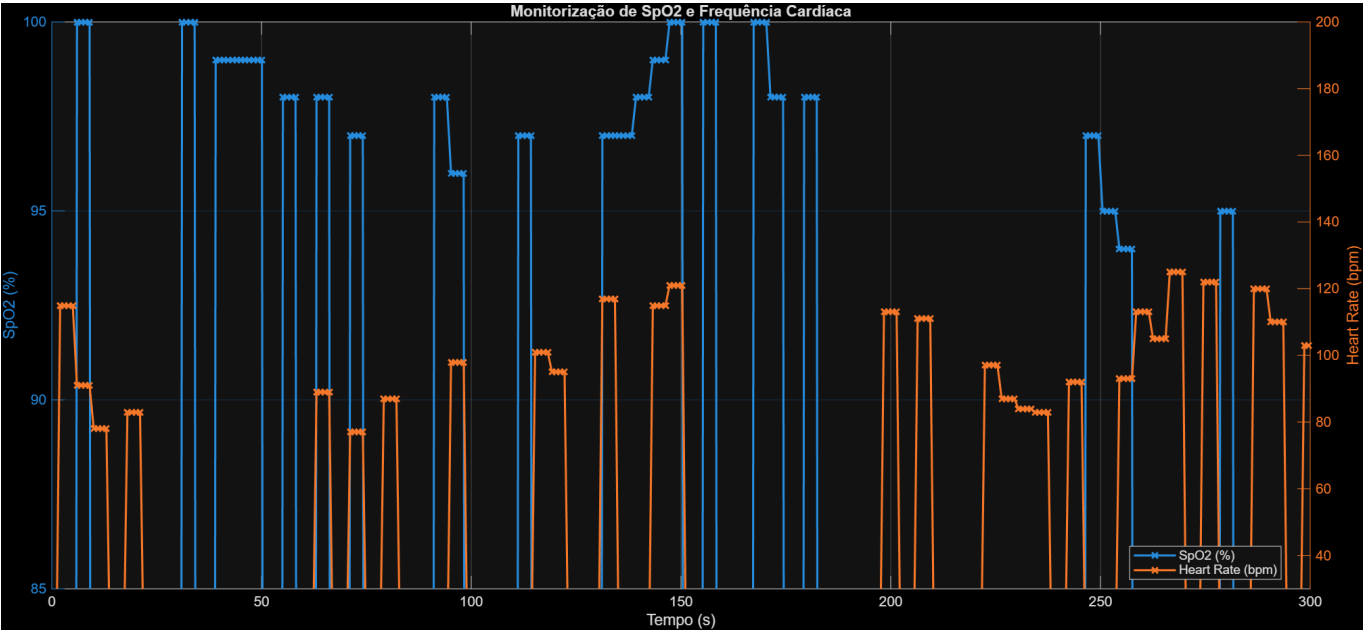
1_0 - 1 min



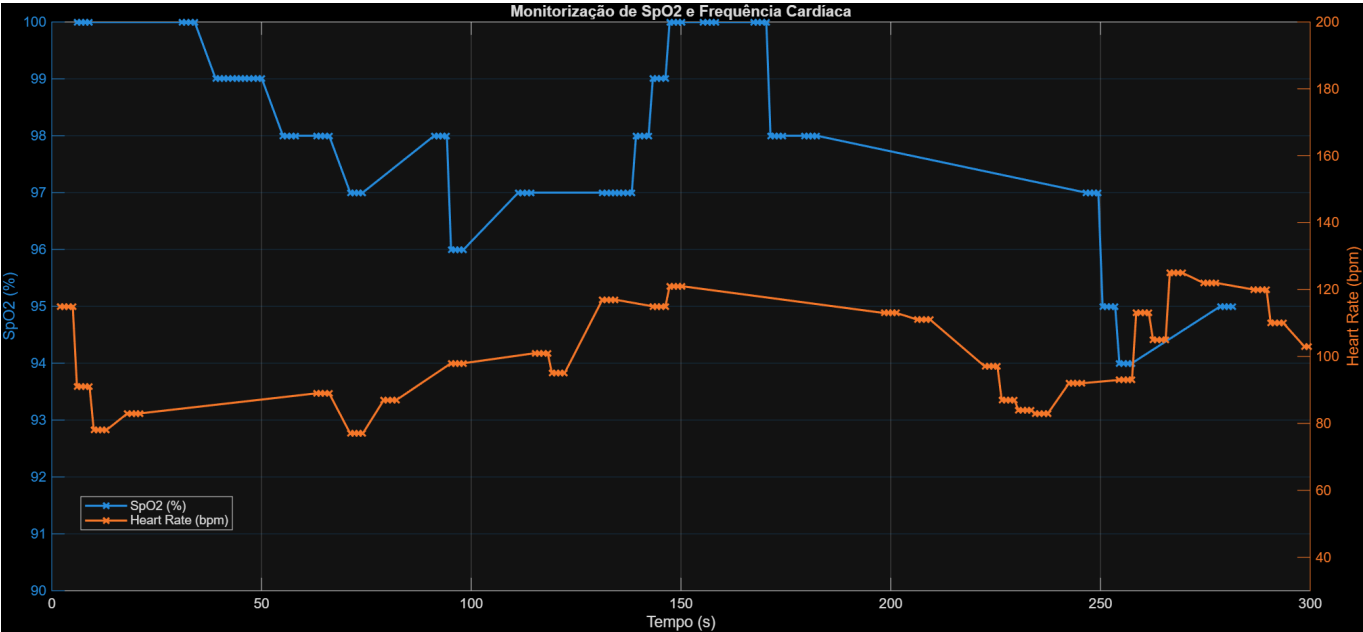
1_1 - 1 min



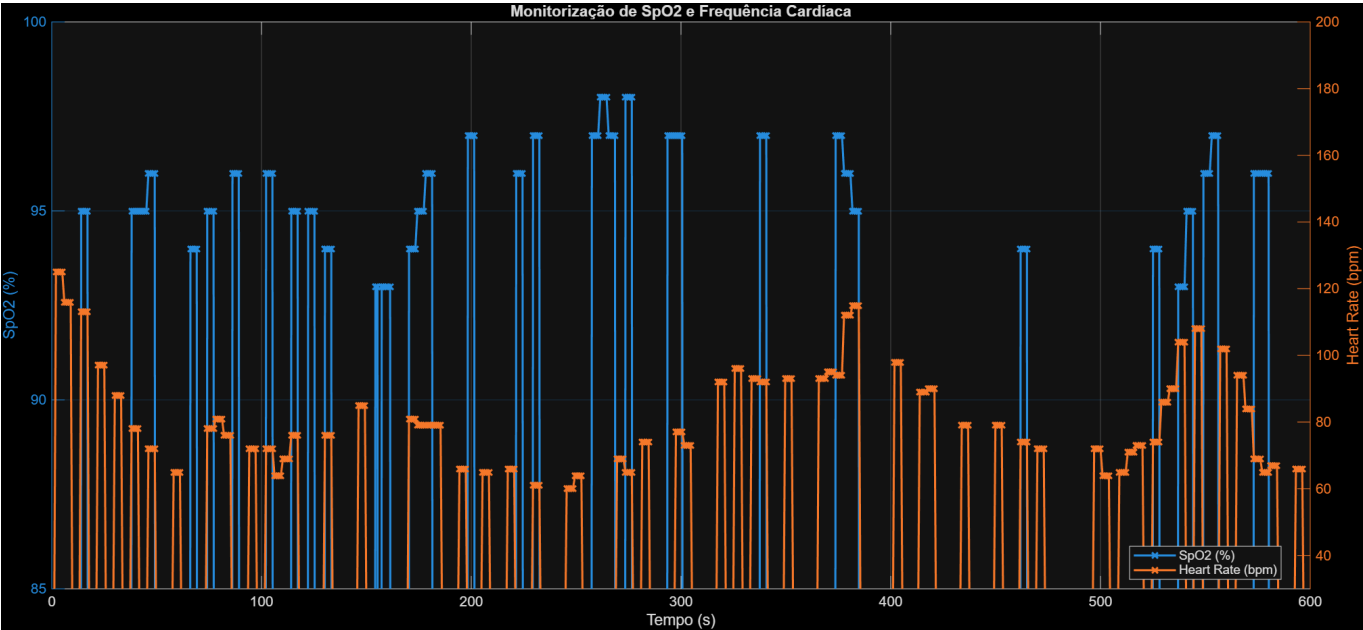
2_0 - 5 min



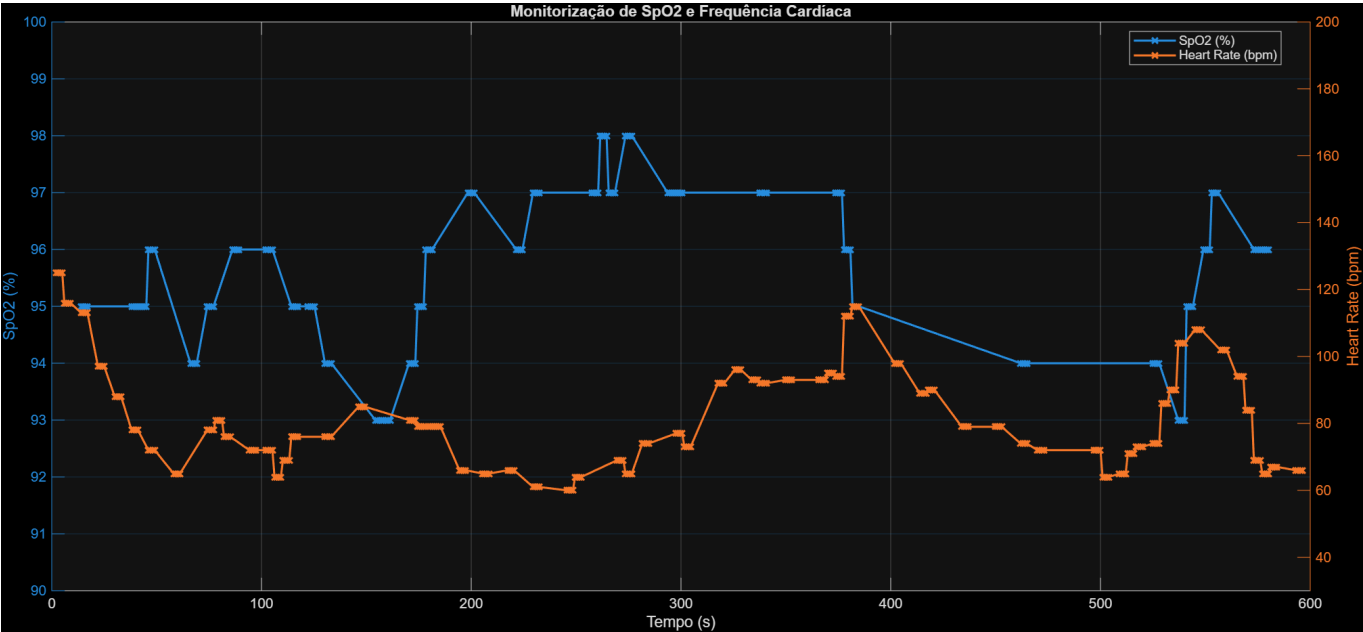
2_1 - 5 min



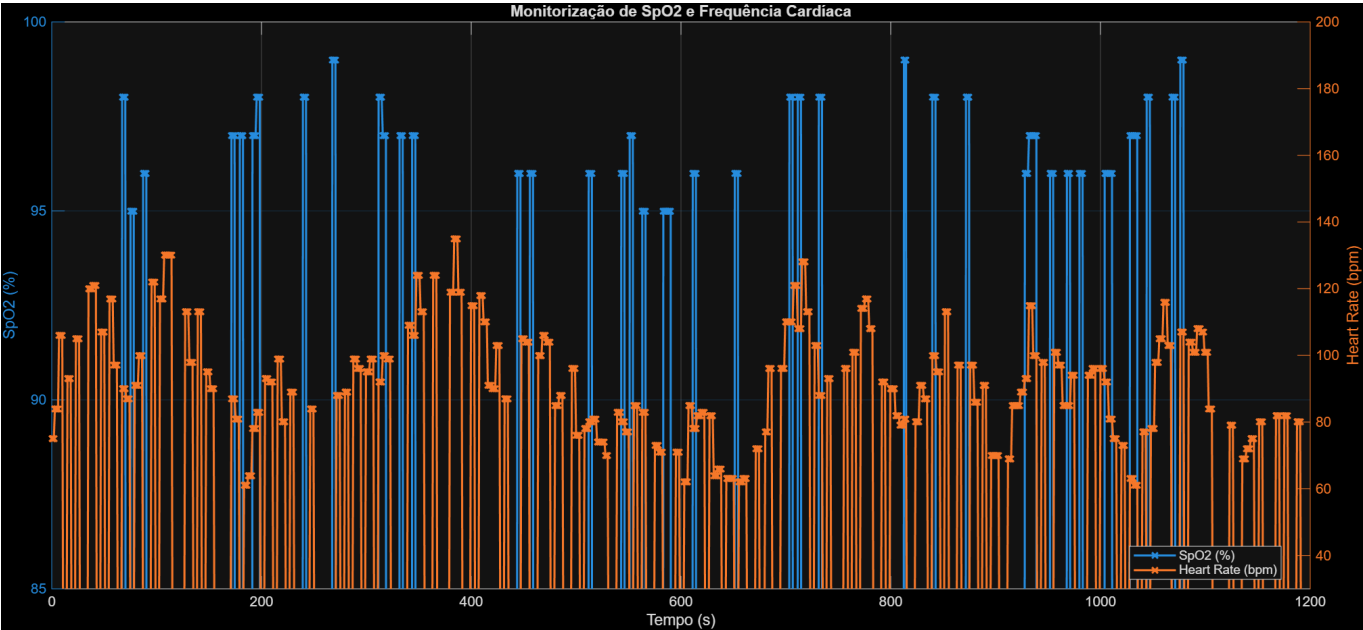
3_0 - 10 min



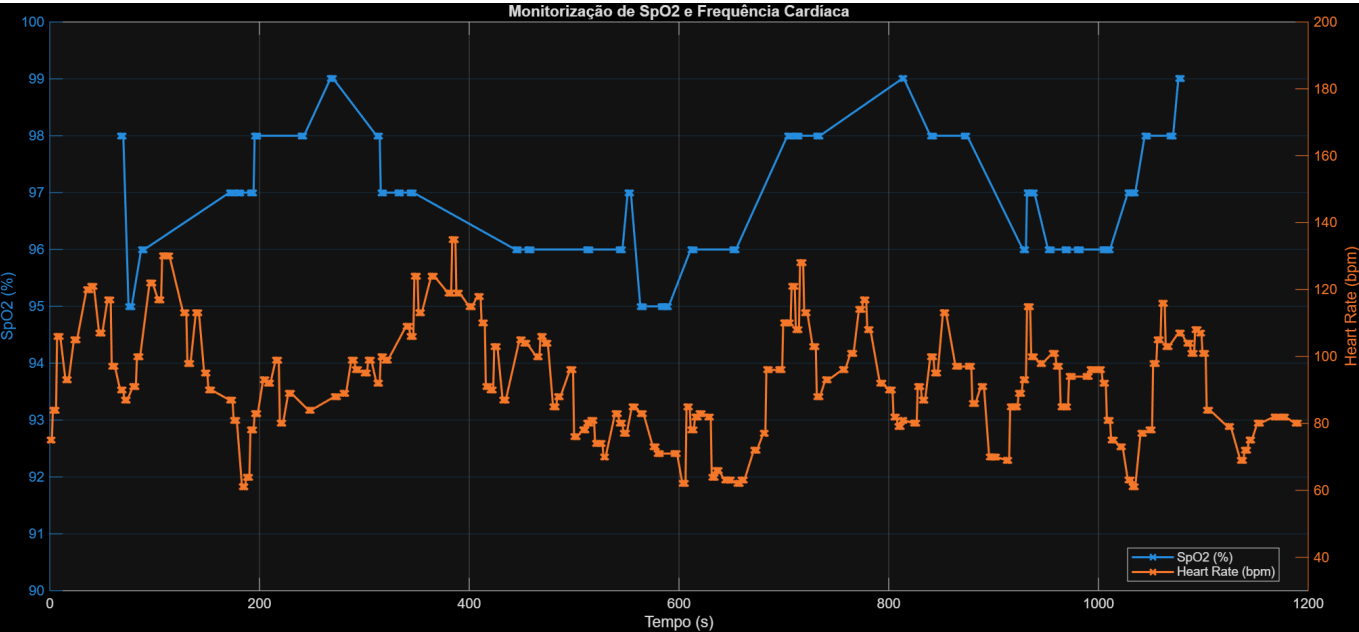
3_1 - 10 min



4_0 - 20 min



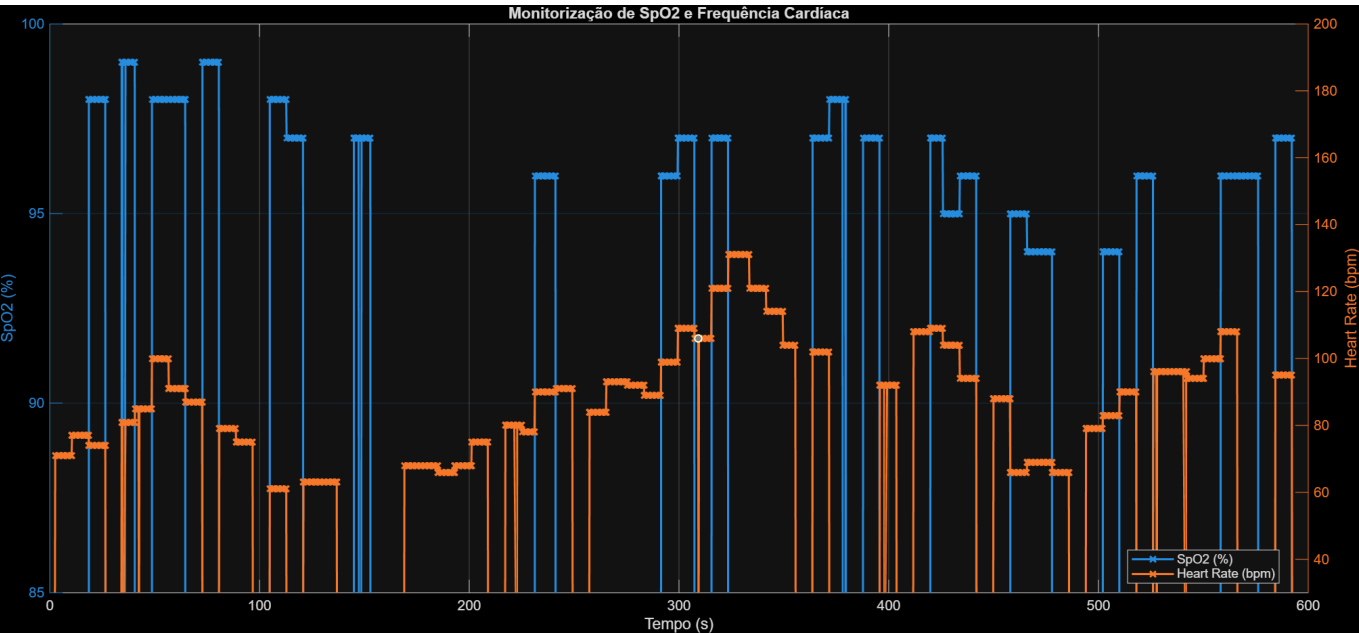
4_1 - 20 min



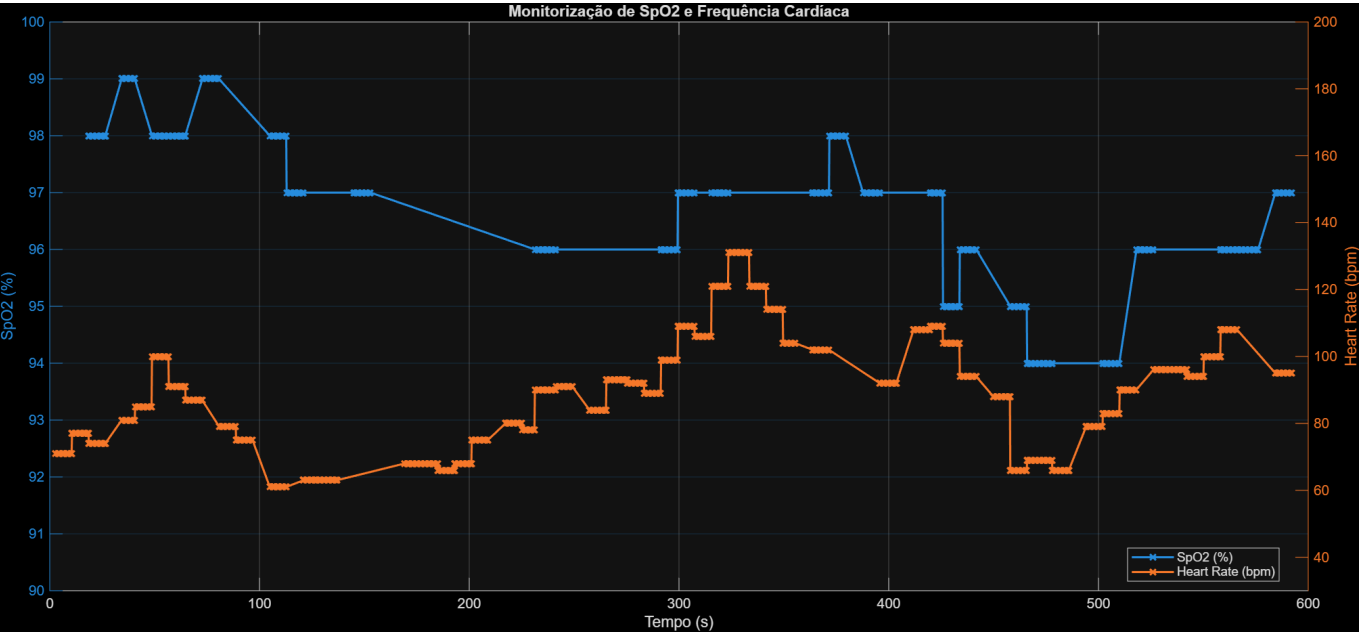
Gráficos com duty-cycle 50%

Esterno, testes com a duração de 10 min, entre cada teste a esp32 foi reiniciada Medidos numa mulher de 20 anos saudável, em repouso, para todos os valores destes testes foram guardados Neste teste, o sensor liga, tira dados de 200ms em 200ms e desliga, assim em loop.

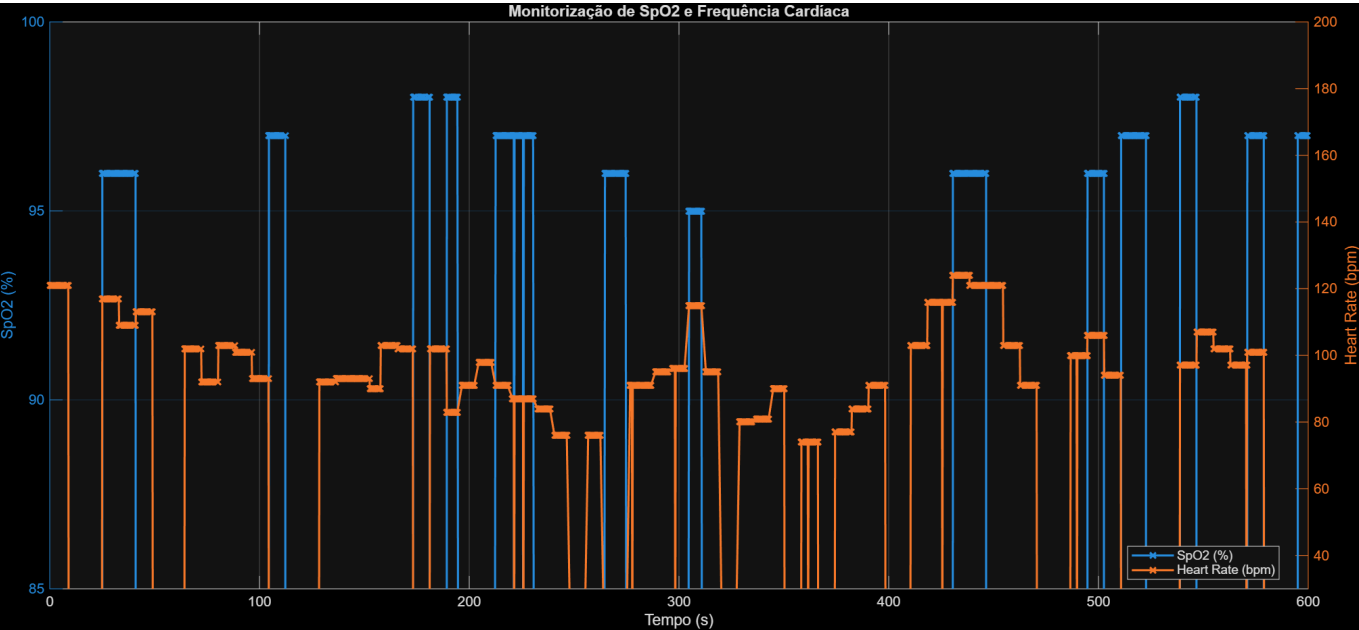
5_0 (1s ON, 1s OFF)



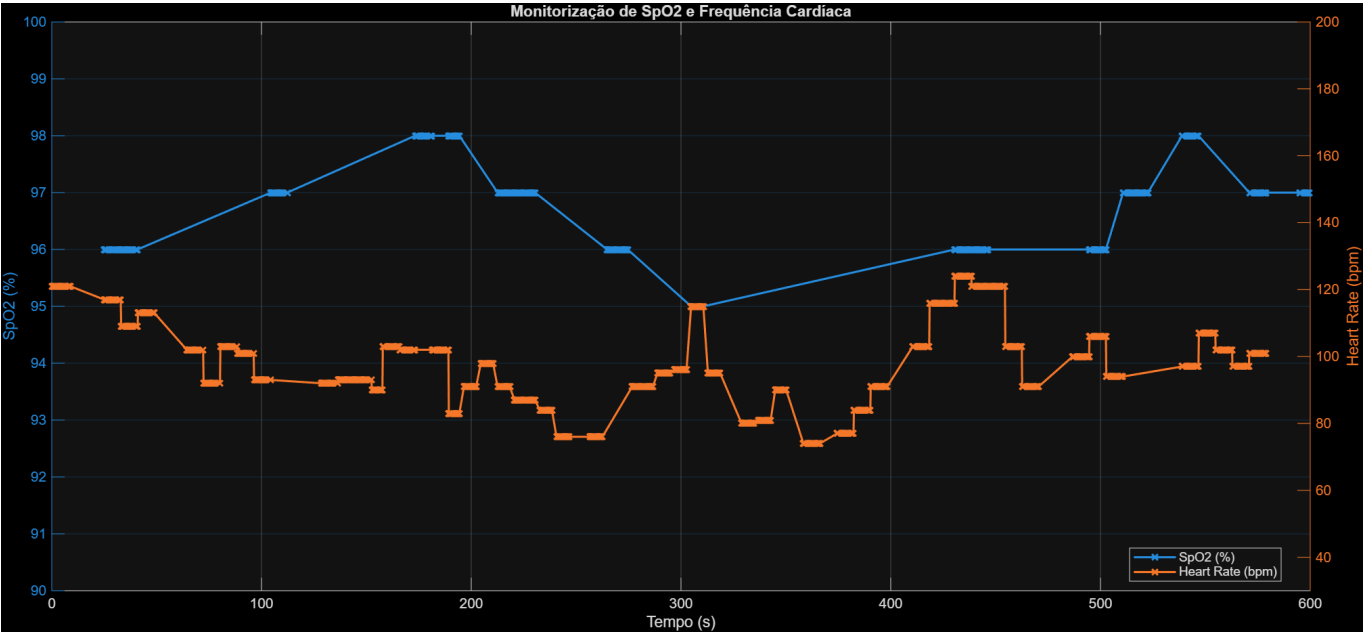
5_1 (1s ON, 1s OFF)



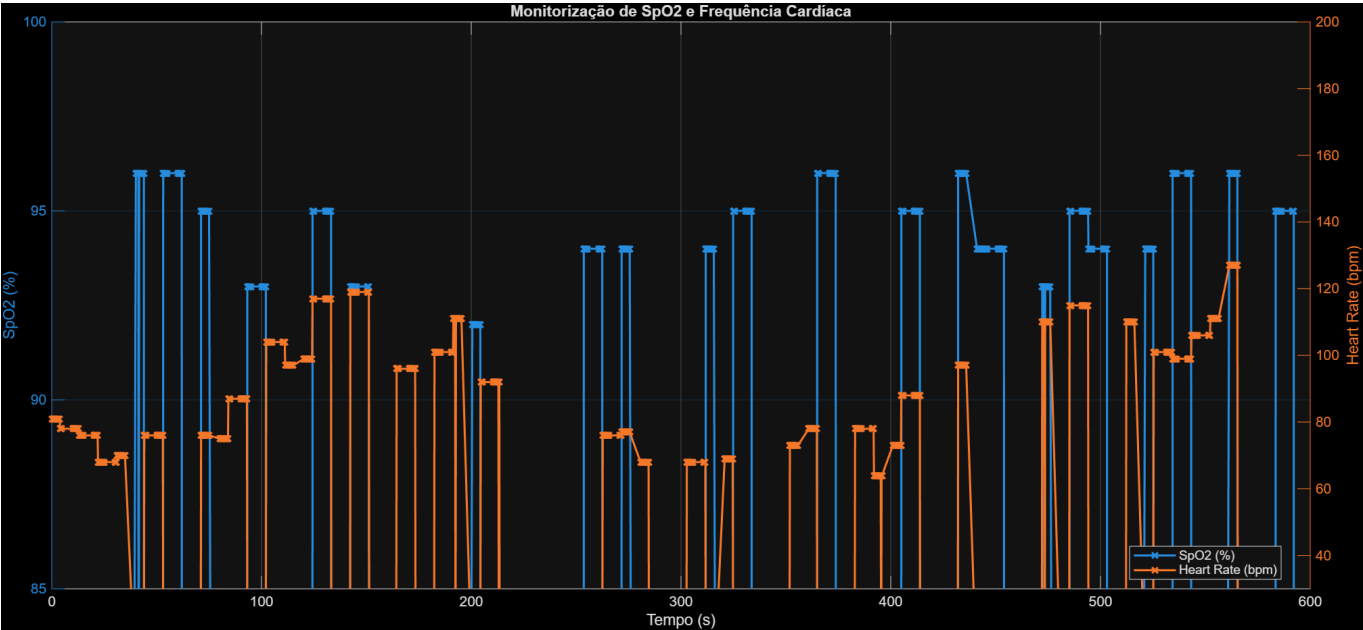
6_0 (2s ON, 2s OFF)



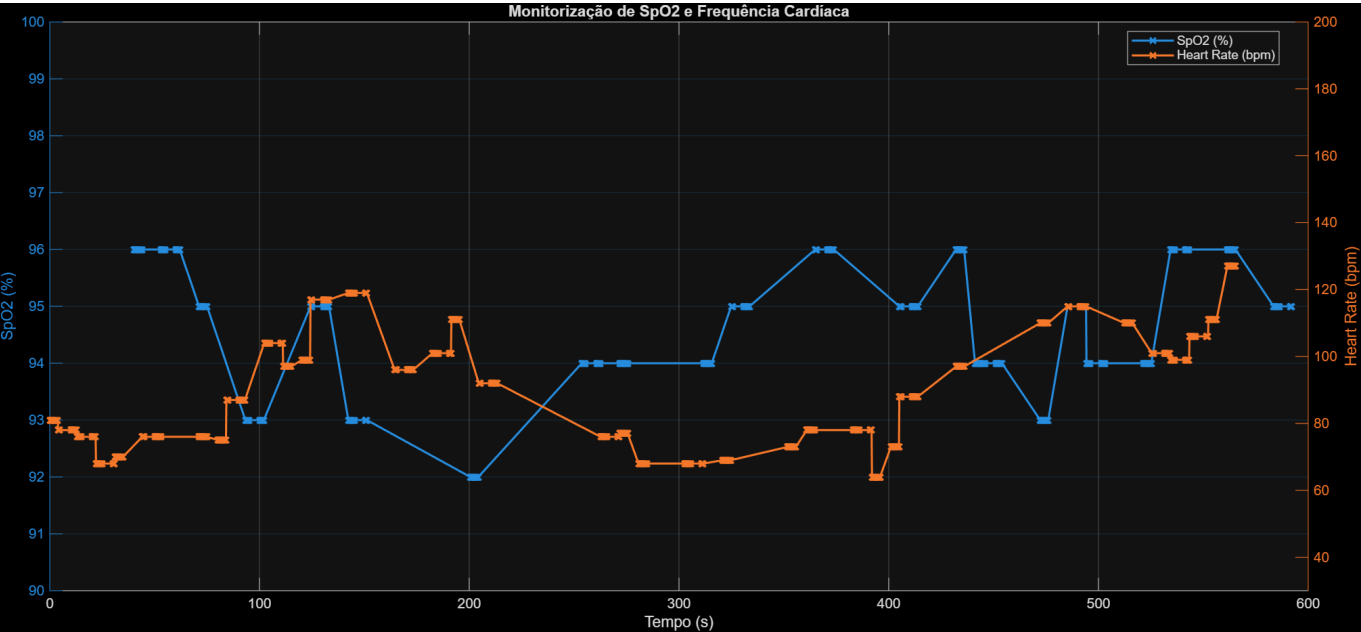
6_1 (2s ON, 2s OFF)



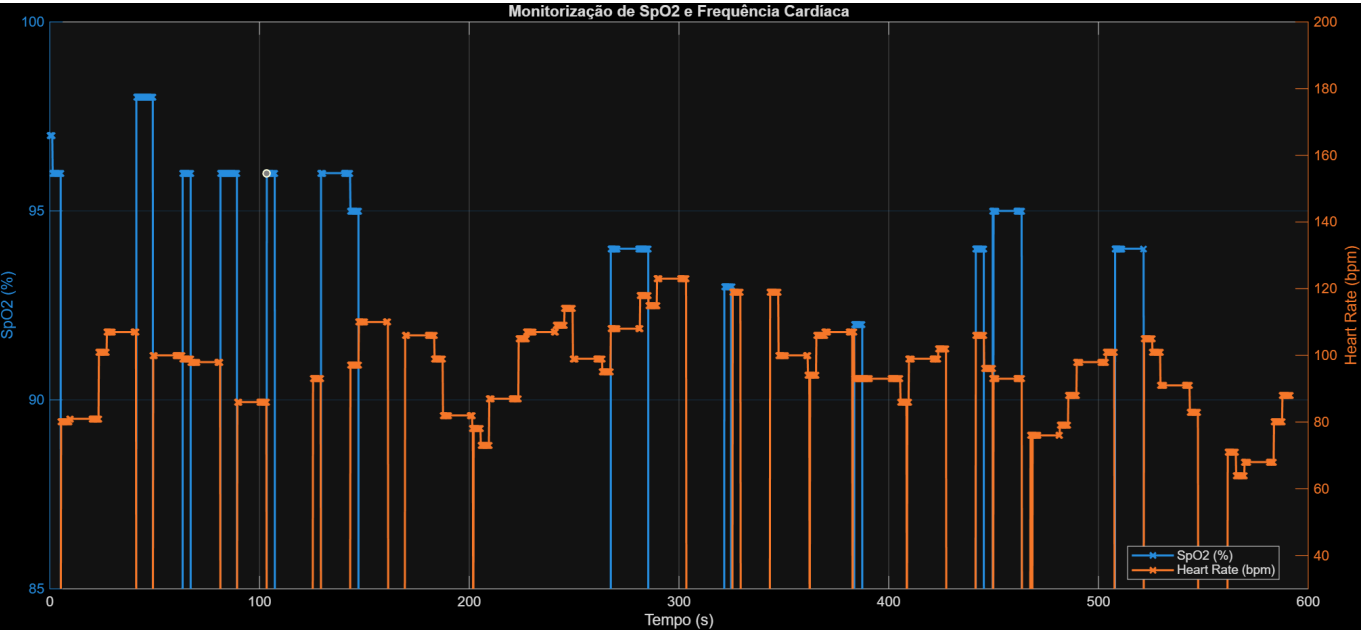
7_0 (5s ON, 5s OFF)



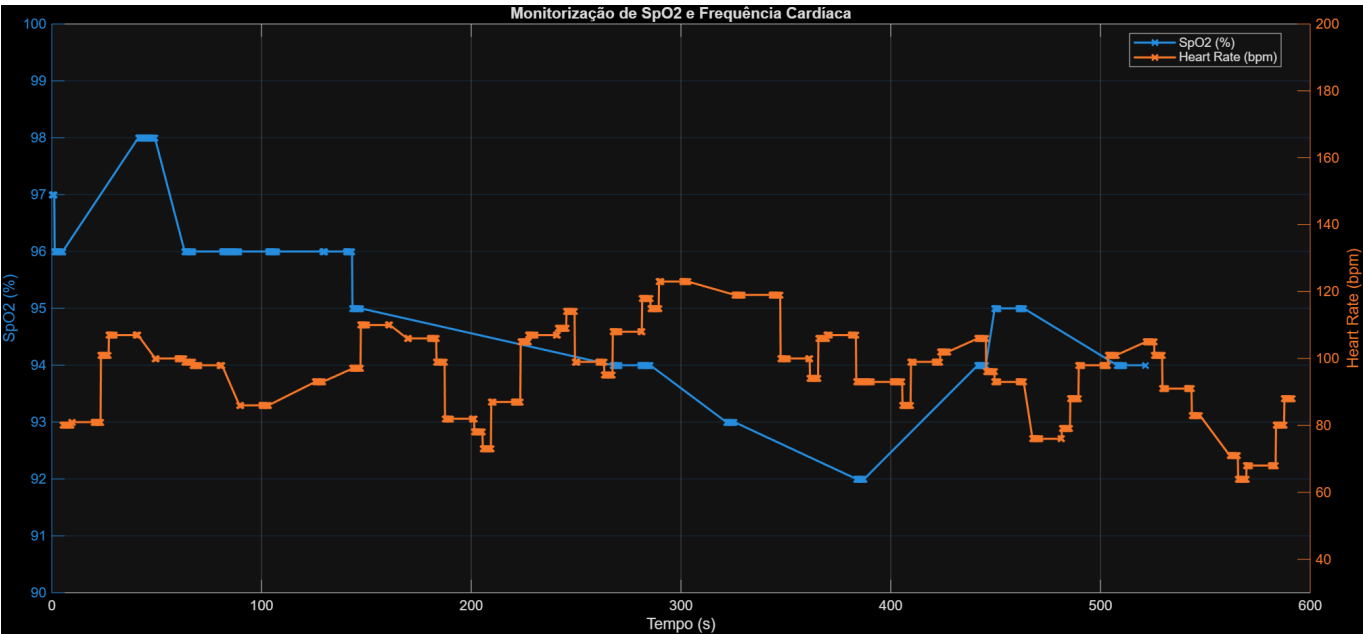
7_1 (5s ON, 5s OFF)



8_0 (10s ON, 10s OFF)



8_1 (10s ON, 10s OFF)



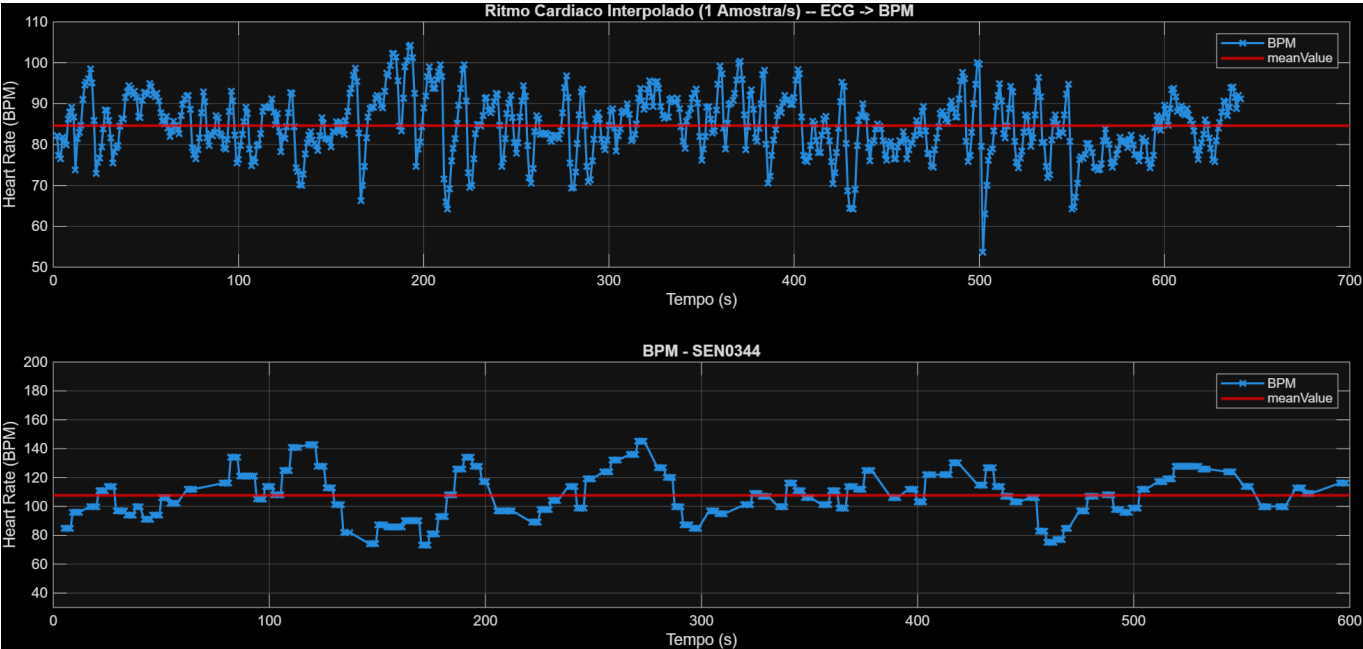
TESTAR OUTROS TESTES COM DUTY CYCLES DIFERENTES DE 50%

Gráficos comparação no IEETA

Nestes testes foram comparados dados extraídos através de um ECG que foram filtrados para obter o BPM que foram comparados com o BPM obtido pelo SEN0344, sempre de 1 em 1s (o computador estava a carregar)

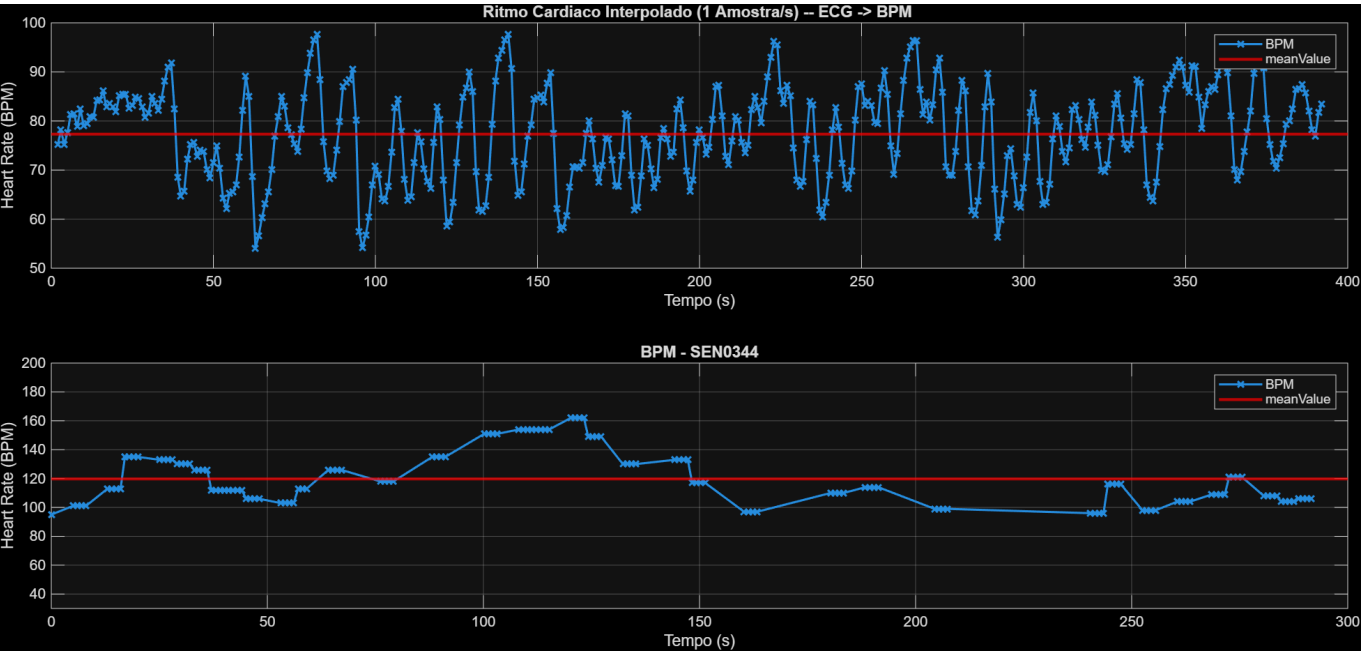
IEETA1_1 VS ECGtoBPM1

Almeida parado a falar 10min medições 1 Ecg guardado como ecg1



IEETA2_1 VS ECGtoBPM2 - BASE LINE

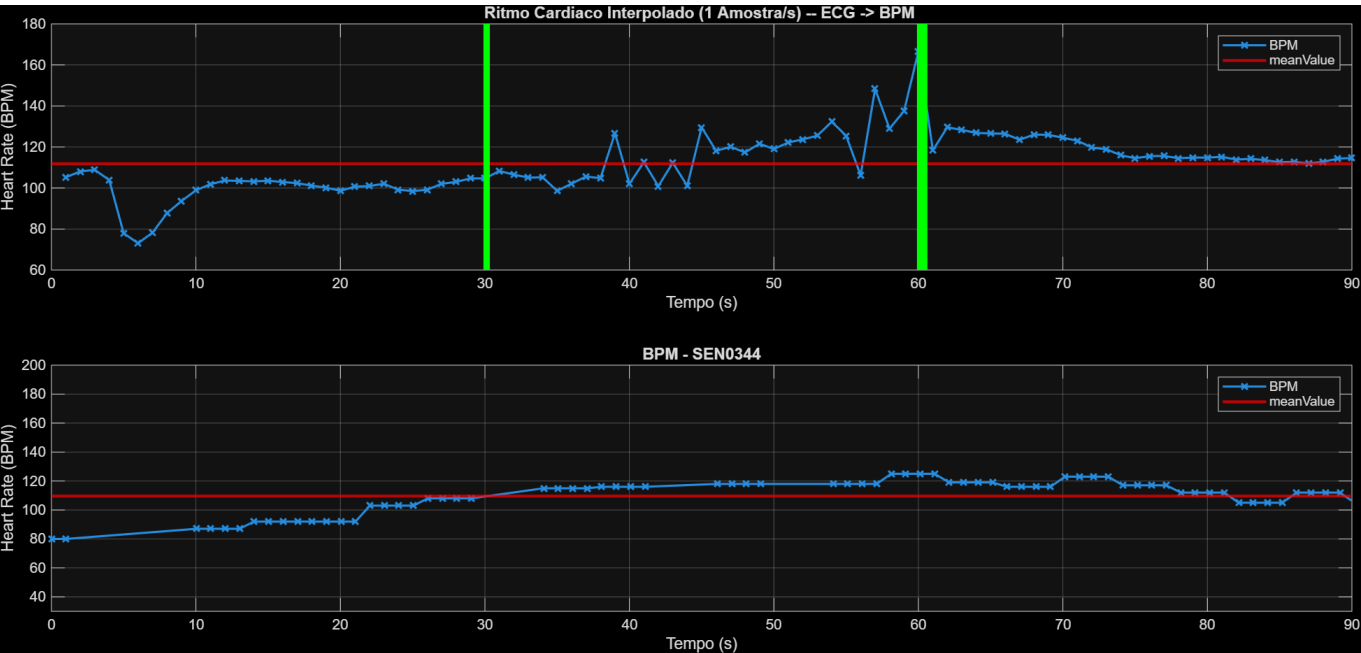
Almeida - 5 min - totalmente parado Ecg guardado como ecg2



Testes em movimento ieeta

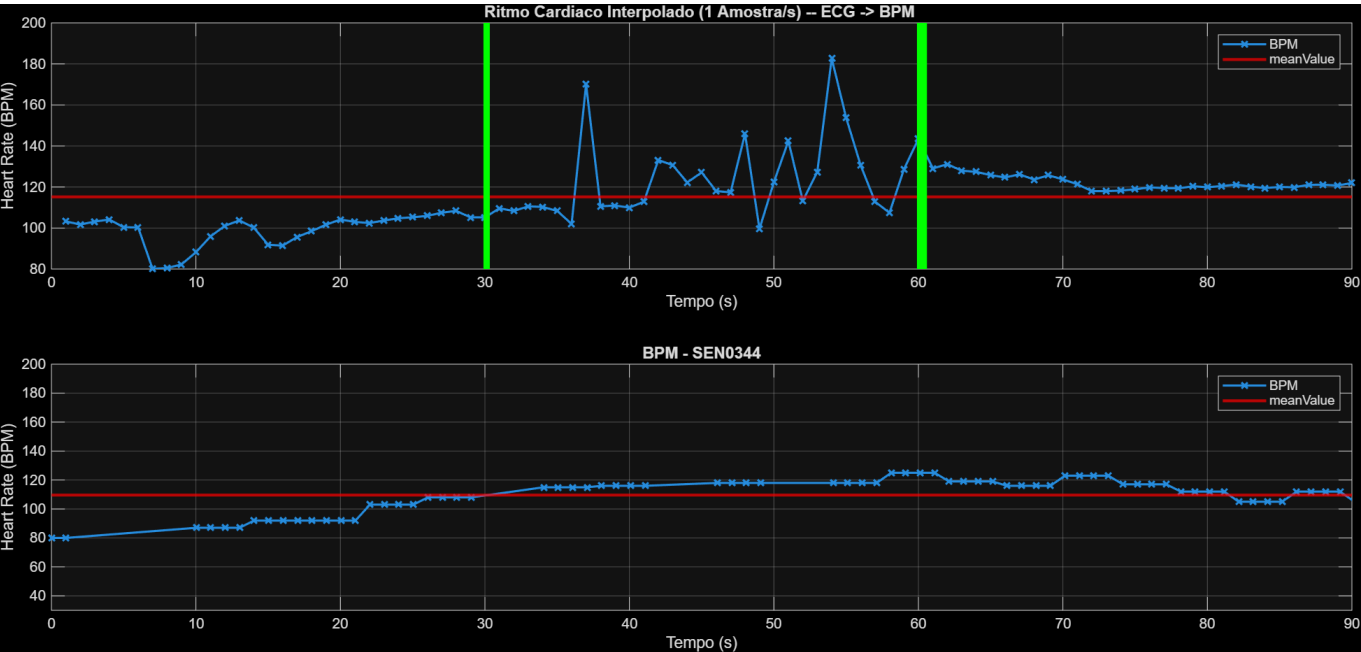
30s parado, saltar 30s, 30s parado TOTAL 90s

IEETA3_1 VS ECGtoBPM3 pc sem estar ligado à corrente IEETA3



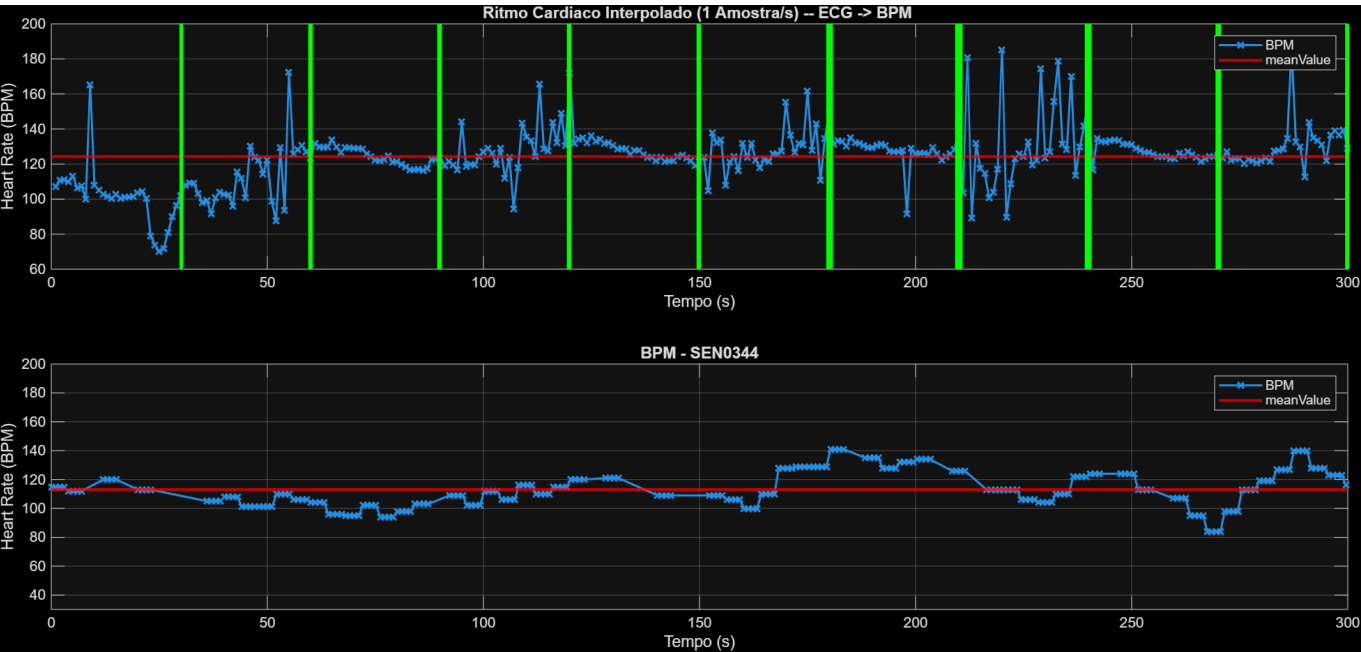
30s parado, saltar 30s, 30s parado TOTAL 90s

IEETA4_1 VS ECGtoBPM4 pc sem estar ligado à corrente a segurar no sensor IEETA4



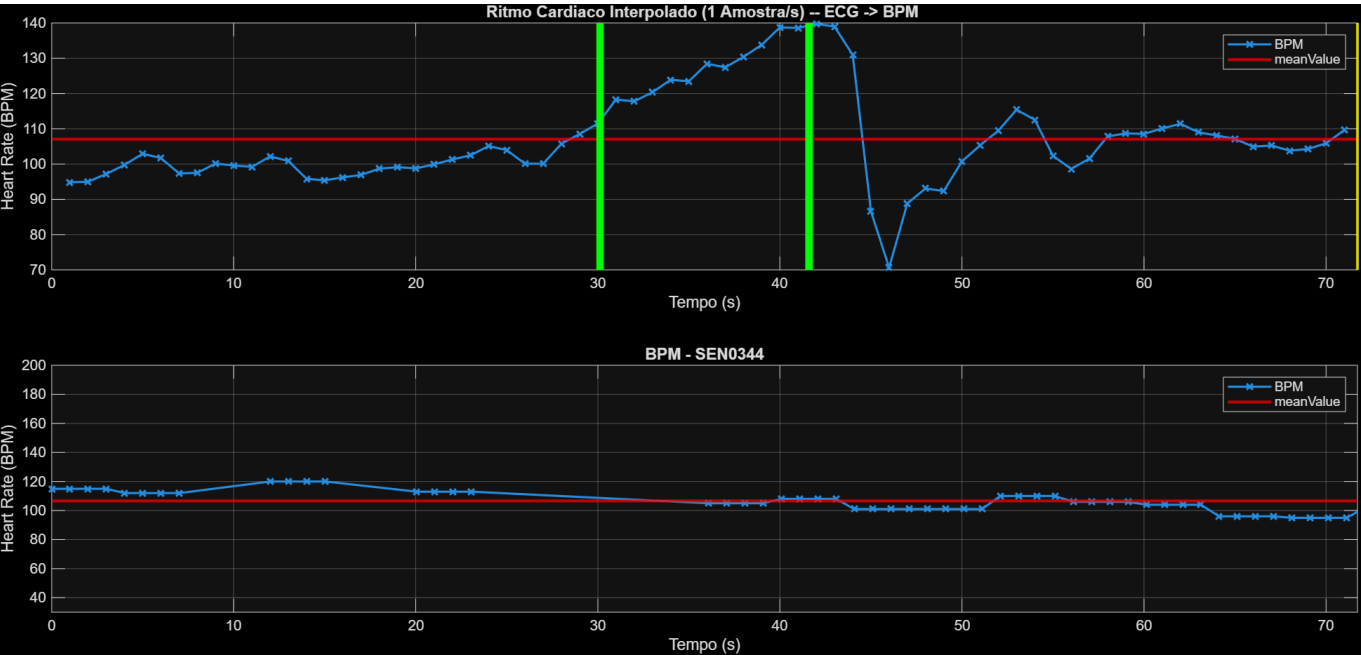
30s parado, saltar 30s, 30s parado TOTAL 5min

IEETA5_1 VS ECGtoBPM5 IEETA5



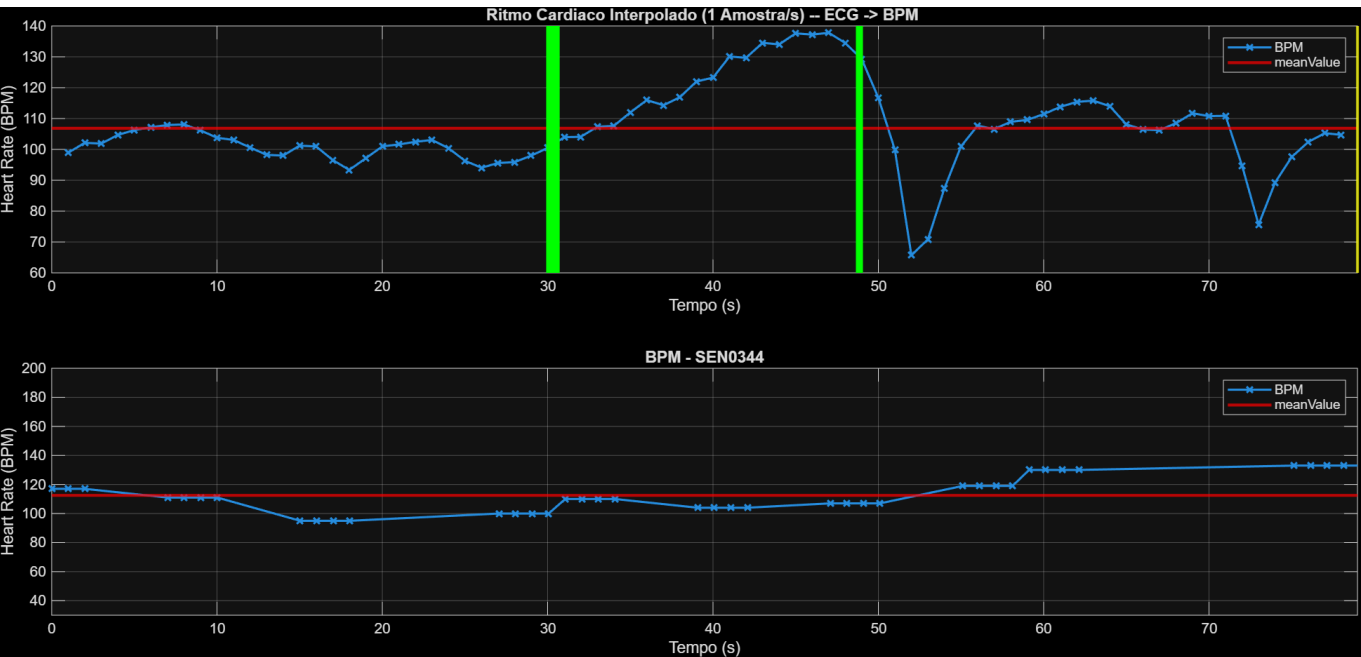
repouso 30s, 5 flexoes, repouso 30s

IEETA6_1 VS ECGtoBPM6 IEETA6



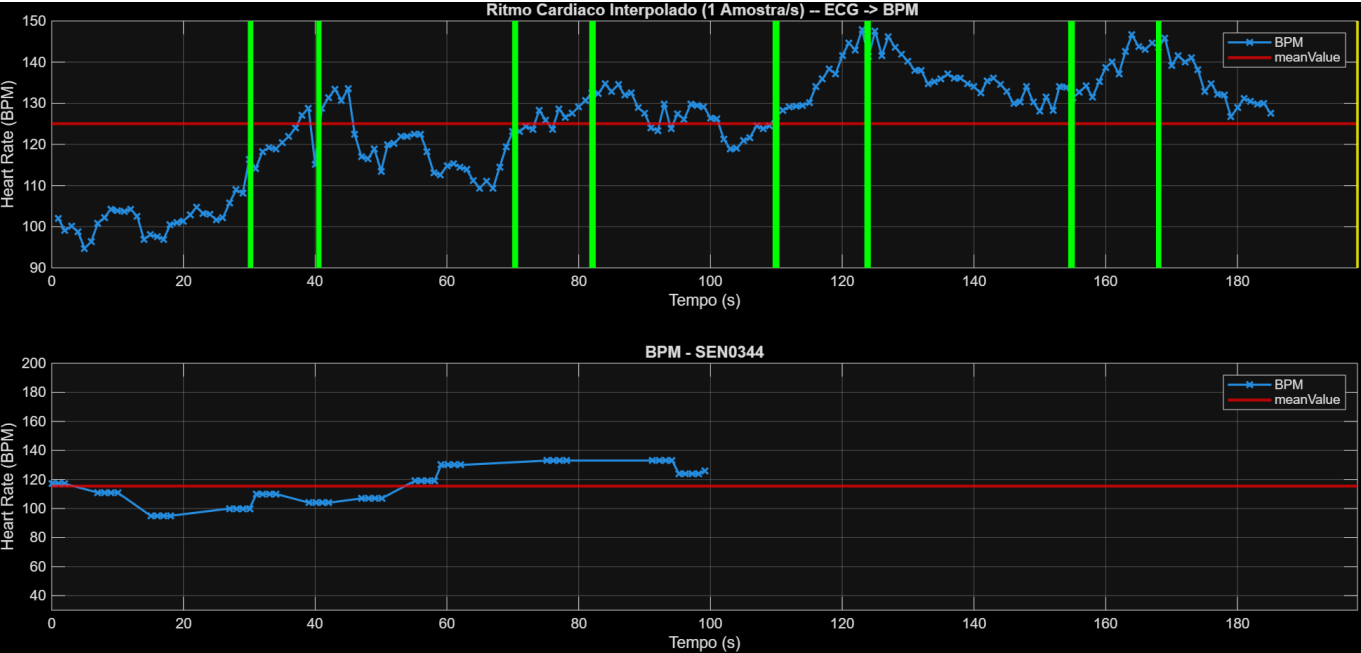
repouso 30s, 5 flexoes, repouso 30s

IEETA7_1 VS ECGtoBPM7 IEETA7



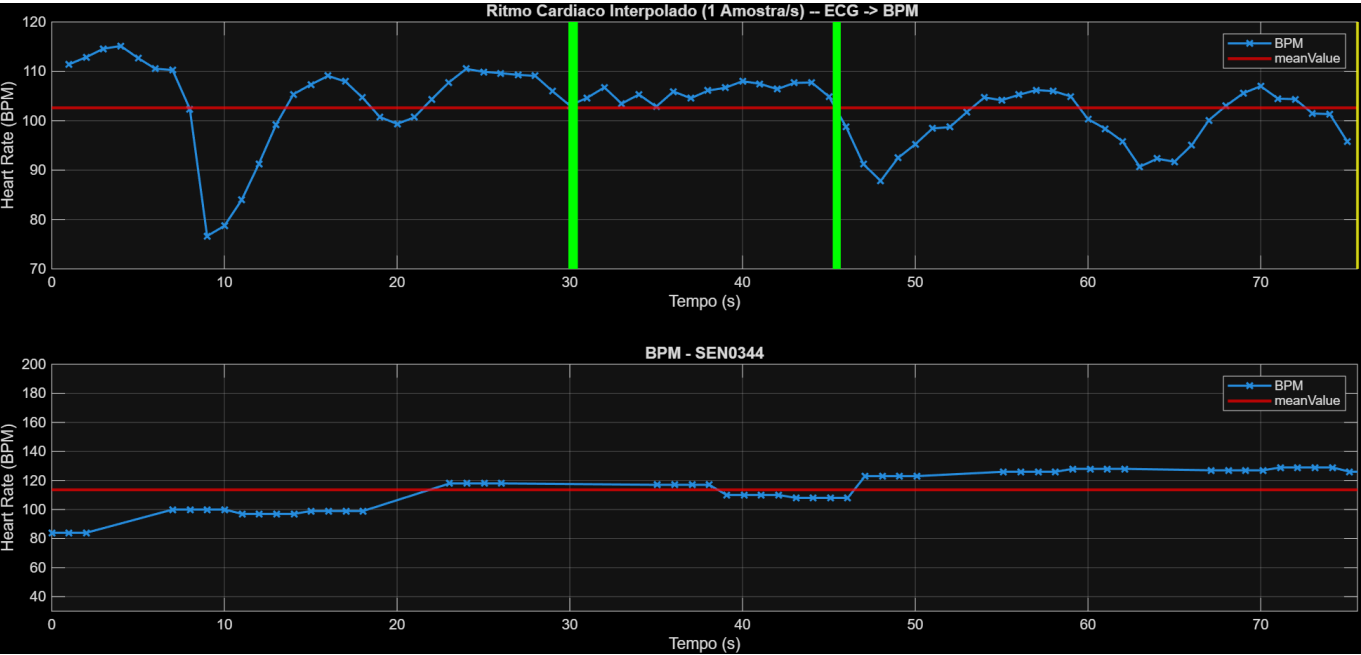
30s parado, 5 flexoes, 30s parado TOTAL 5min

IEETA8_1 VS ECGtoBPM8 IEETA8 Nota: não sei oq aconteceu com este teste - teste inválido



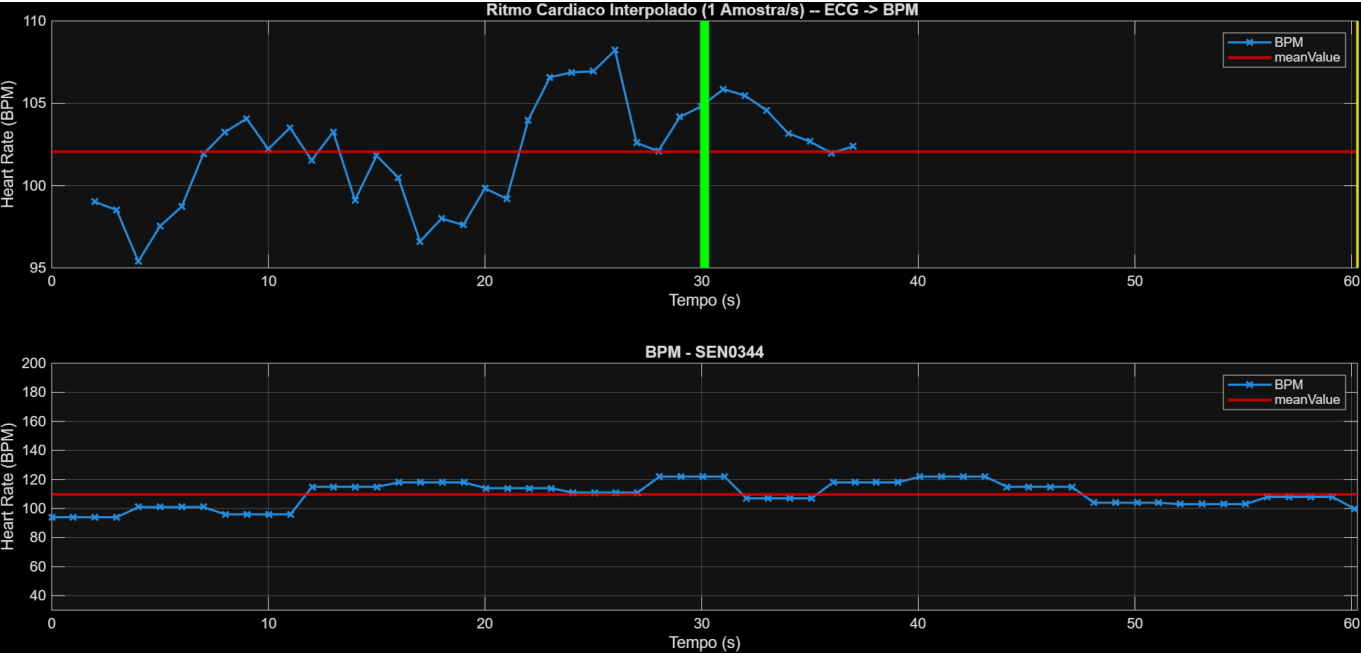
30s repouso 15s bater 30s repouso

IEETA9 IEETA9_1 VS ECGtoBPM9



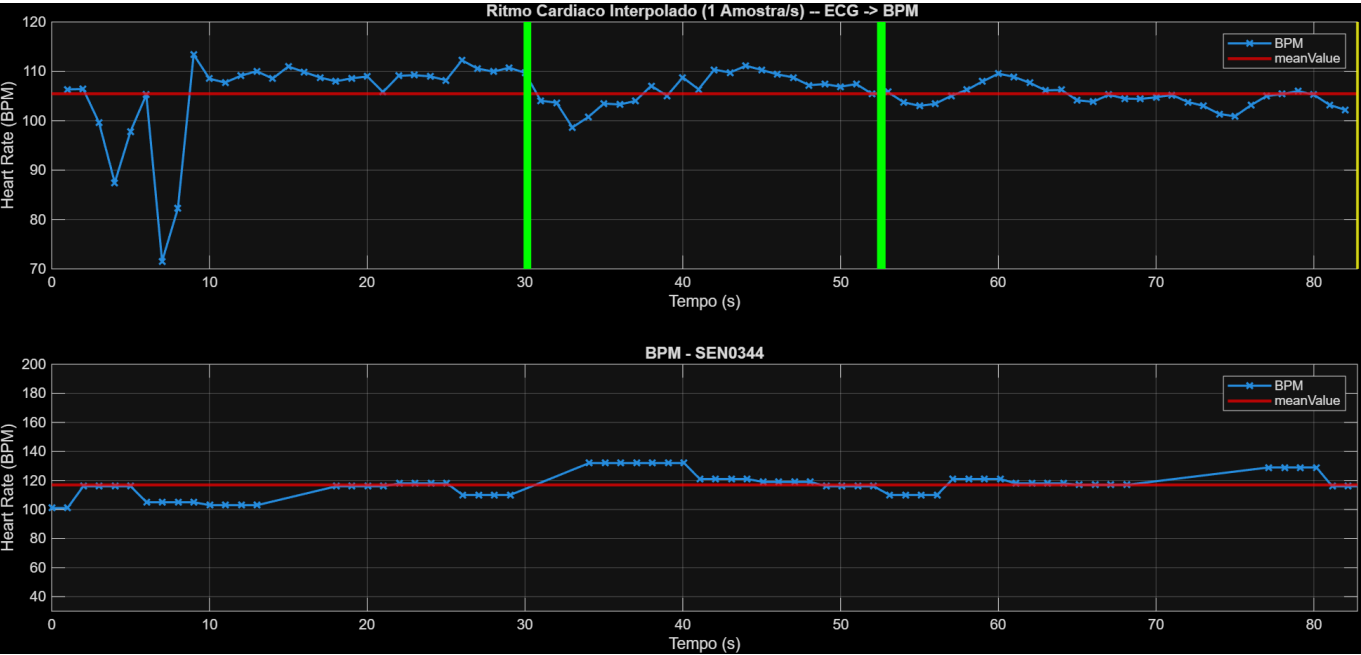
30s repouso 15s bater 30s repouso

IEETA10 IEETA10_1 VS ECGtoBPM10 Nota: não sei oq aconteceu com este teste - teste inválido



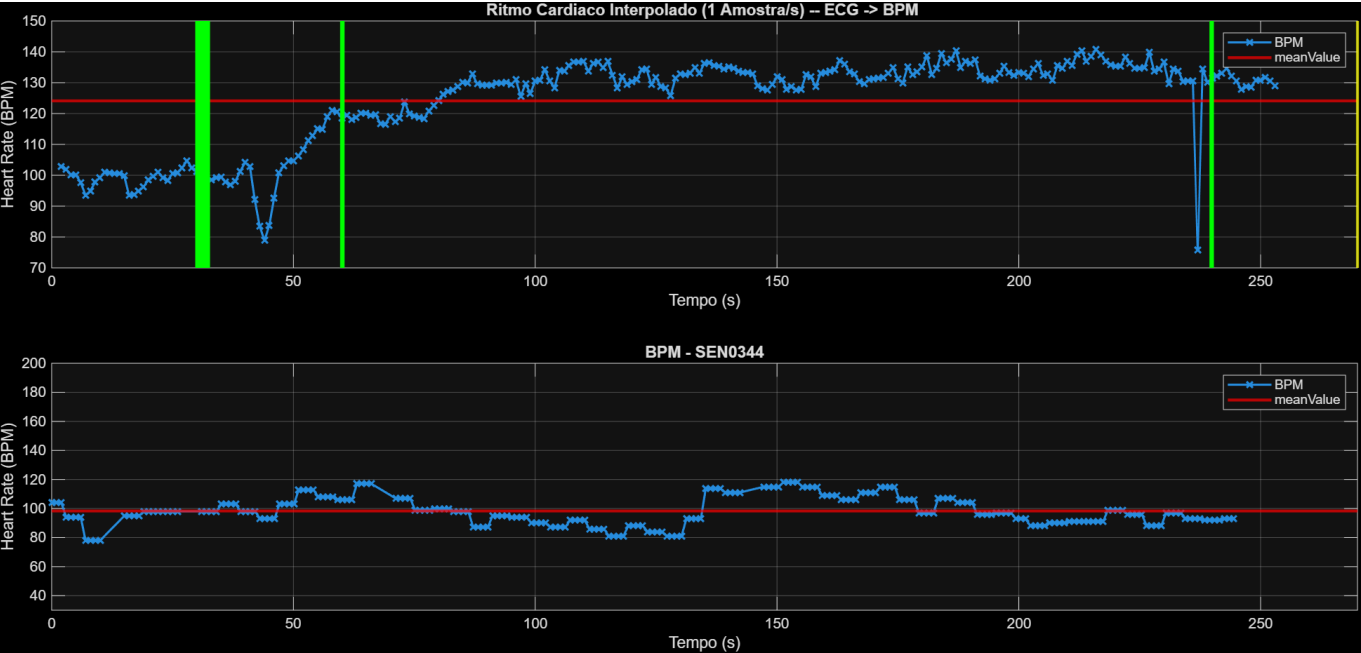
30s repouso 15s bater 30s repouso

IEETA11 IEETA11_1 VS ECGtoBPM11



30s repouso 3min andar 30s repouso

IEETA12 IEETA12_1 VS ECGtoBPM12



30s repouso 3min andar 30s repouso

IEETA13 IEETA13_1 VS ECGtoBPM13

